CS110 / INDEPENDENCE ROAD

team / Lexxie

Issued: Nov 10, 2026, 2:50 PM Pacific

Record: FORM-2935112442B32DC30DC60A17

Personal public verification code: VIEW-9265F91D9B43EF8355AB19A9

Include your own public code with every assignment, even in a team folder. Never submit your private recovery key.

SHA-256: 90e521928e5e9e46c5c0e748bbccb4876e5806a63e544a5c49054394780b721c

Verify the record number at courses.addp2p.com/development-road/independence/index.html?view=verify

An issued record preserves submitted information. It is not a grade or proof of comprehension.

team

TEAM-E9DD8E3D6C7E176C03D956F1

Name

L3X1C0N_WRLD

lead

Lexxie

Team members

0

Name

Lexxie

verificationCode

VIEW-9265F91D9B43EF8355AB19A9

Preparation record

PREP-F39AC39A936F4F003C2A6D22

Contribution record

FORM-A91B6BD4CBEDB5F72519F72E

timeA

Time code

TIME-096E904E83B4BD764B9404FD

Name

Lexxie

Action

A

Server arrival

2026-11-10T21:02:47.141Z

Class starts

2026-11-10T20:45:00Z

Calculated late points

15

Late deduction points

15

CS110 / INDEPENDENCE ROAD

Review status

Calculated awaiting instructor review

Attention Accord plan

mission

I want to make a beautiful playable scene in unity with a 3D player viewed in 3rd person.

framework

I will use the AAA framework in order to keep an organized structure of how I am able to complete my project efficiently.

pipeline

Unnamed stage

Unnamed stage

I used mermaid owner Lexxie

Mermaid worked better for me.

Production Pipeline Explanation: StimuLye Vertical Slice (AAA Framework)

Our production pipeline applies Dr. Malik Stalbert's AAA Framework (Application Action Amplification) to transition our team smoothly from an initial template into a playable, narrative-driven prototype:

1. Stage 1: Application (Setup & Baseline)

Following the principle that "application gets them moving," we do not start by writing code from scratch. We establish our shallow directory structure (D:\PROJECTS\StimuLye) to bypass Windows path length limits and deploy Unity's verified Third Person Core template. This gives us immediate physics, camera orbit, and movement controls within 30 minutes.

2. Stages 2 & 3: Action (Mechanics, Avatars & Environments)

Once the baseline is alive, the team enters the "Action" phase, where concepts and custom assets are introduced:

- Character Integration: We import our NaN model, set its rig to Humanoid, safely unpack the player armature, and swap the geometry. The Humanoid avatar acts as the bridge so existing template animations retarget cleanly to our custom protagonist.
- Environment Construction: Our level designer uses modular prefabs (the "cookie-cutter" approach) to build the looping hospital ward and dark memory rooms without wasting memory.

3. Stage 4 & 5: Amplification (Core Loop, Polish & Verification)

In the "Amplification" phase, private technical competence expands into an integrated experience:

- Core Narrative Mechanic: We implement the "Stimuli" interaction script. Picking up the sensory object triggers a room material swap from dark void to stable hospital textures while playing the first "Static" audio cue.
- Verification & Team Review: The team conducts an end-to-end playtest, verifying that the opening dark sequence, corridor movement, and memory stabilization run smoothly with zero console errors.

Roles & Time Estimates:

- Block 1 (30m): Technical Lead (Template & Directories)

CS110 / INDEPENDENCE ROAD

- Block 2 (45m): 3D Artist / Character Lead (Rigging & Retargeting)
- Block 3 (60m): Level Designer (Modular Ward Prefabs)
- Block 4 (60m): Gameplay Programmer & Audio Lead (Stimuli Triggers)
- Block 5 (45m): QA & Entire Team (Playtest & Milestone Sign-off)

Total Sprint Duration: 4 Hours (240 Minutes)

tech

What I Did / Decided / Observed:

As a solo developer on StimuLye, my priority is eliminating pipeline friction so I don't get overwhelmed wearing every hat alone. I applied the TnT ("Tool and the Temple") principle: every tool must serve my MVP vertical slice and reduce manual labor. I chose a lean, verified stack built on Unity 6.6 and established personal testing gates to check my own work at each milestone.

My Agreed Solo Tool Stack & Verification Plan:

Engine: Unity 6.6 (6000.6) with URP

Why: Faster Play Mode entry, lighter memory footprint, and native Universal Render Pipeline lighting/fog to achieve StimuLye s dark atmosphere without third-party render plugins.

Who Checks It: Me (wearing the Technical Hat) verified via build tests and frame rate checks.

Language & IDE: C# / Visual Studio Community

Why: Standard Unity scripting for clean, fast implementation of the "Stimuli" interaction and room material swap trigger scripts.

Who Checks It: Me (wearing the Programmer Hat) verified with zero console errors and clean syntax.

Base Template: Starter Assets Third Person Controller + Cinemachine

Why: Provides an instant movement, sprint, jump, and camera follow baseline. As a solo dev, this saves me dozens of hours of physics and camera programming so I can focus directly on the story mechanics.

Who Checks It: Me (wearing the Technical Hat) verified through Play Mode controller feel.

Art & Rigging: Mixamo + Blender

Why: Mixamo automatically generates the Humanoid rig and Avatar for my custom "NaN" character; Blender is used for quick adjustments and whiteboxing modular ward prefabs.

Who Checks It: Me (wearing the Visual Hat) verified by checking that animations retarget without bone stretching.

Audio: Audacity / FreeSound (WAV stems)

Why: Simple, lightweight sound editing for hospital room tone, heart monitor beeps, and the "Static" cue.

Who Checks It: Me (wearing the Audio Hat) verified with in-engine volume balance checks.

Progress Tracking: Git + Project_LOG.txt

Why: Local Git commits protect my project from corruption; the auto-timestamped .LOG file holds me accountable to daily tasks.

Who Checks It: Me (wearing the QA / Architect Hat) audited against my GDD checklist.

Question / Self-Check:

How can I automate my daily .LOG entries and Git commits so project tracking remains effortless while working solo?

Time code

Classes & components

Class | Responsibility | GameObject

ThirdPersonController | Processes movement calculations, sprint, jump physics, gravity, and drives animator parameters. | PlayerArmature

StarterAssetsInputs | Reads raw input data (WASD, mouse delta, space, shift) from the Input System and stores state flags. | PlayerArmature

CharacterController | Unity engine physics component that manages capsule collision, ground detection, and slopes. | PlayerArmature

StimulusItem | Detects player proximity, triggers interaction events, and signals memory recovery when collected. | Stimulus_MemoryObject

RoomStabilizer | Listens for Stimulus collection; swaps room shaders/materials from dark void to hospital textures. | Environment_Manager

StaticController | Monitors player distance to boundaries; increases audio distortion and visual grain as "Static" stalks. | Static_Antagonist

PauseMenuController | Listens for the Esc key; pauses time and displays the CRT power-down / heart monitor scene. | UI_Canvas_Manager

Variables / properties

Name : type | Value / purpose | Used by

MoveSpeed | Defines base walk velocity vs. sprint speed in the ward. | ThirdPersonController

_animIDSpeed | Cached hash IDs for fast Animator parameter updates. | ThirdPersonController

isStabilized | Prevents the room transformation from firing more than once. | RoomStabilizer

darkMaterial | References to the low-poly void shader vs. lit hospital texture. | RoomStabilizer

staticProximityThreshold | Distance from player before static distortion audio begins. | StaticController

stimulusSound | Audio stems played on memory pickup and enemy approach. | AudioManager

f(x) Functions

Function | Inputs result | When called

Awake() | Engine Initialization | Caches component references (GetComponent<Animator>(), CharacterController) before scripts run.

Start() | First Frame | Sets the opening scene state: turns off ward lights, applies black void materials, locks cursor.

Update() | Every Frame | Polls player movement vectors, updates camera angles, and checks for the Escape key input.

OnTriggerEnter(Collider) | Physics Collision Event | Detects when NaN steps into the StimulusItem radius or enters a "Static" trigger volume.

OnFootstep(AnimationEvent) | Animation Event | Plays footstep sound cues synchronized with the walk/run animation clip frames.

Packages / using

Package / namespace | Why included | Tested

using UnityEngine; | The fundamental foundation of all Unity code. Provides access to MonoBehaviour, GameObject, Transform, Vector3, Time, and core physics engine types. | [x]

using UnityEngine.InputSystem; | Replaces the legacy Input.GetKey system. Required in Unity 6 for cross-platform event-based input handling (keyboard, mouse look, and gamepads) with zero frame-rate polling lag. | [x]

using Cinemachine; | Controls the dynamic third-person follow camera rig. Automatically handles procedural damping, collision detection with ward walls, and smooth cursor orbital movement. | [x]

using UnityEngine.Events; | Allows decoupled communication via UnityEvent or Action. When NaN collects a Stimulus, the item simply raises an event; the RoomStabilizer and AudioSource respond without being hardcoded to each other. | [x]

using UnityEngine.Rendering.Universal; | Gives C# runtime control over Unity 6's Universal Render Pipeline (URP) volume components (vignette, film grain, chromatic aberration) to create the visual "Static" distortion effect. | [x]

communication

Team check-ins

Description / contact / owner | When Pacific | Agreed

Lexxie | | [x]

reuse

What I Did / Decided / Observed:

Thinking beyond this initial prototype assignment, I audited our production assets to determine what foundational systems can be reused across Act 2 (The Ward), Act 3 (The Deep), and the Finale without rebuilding from scratch. I decided to maintain a strict licensing and attribution log in /Requirements/ so all third-party credit requirements are tracked before scaling the game.

Reusable Components, Sources & Credit Requirements:

CS110 / INDEPENDENCE ROAD

Core Character Controller & Camera Rig:

What is Reusable: The PlayerArmature with Third-Person movement physics, jump/sprint logic, and Cinemachine orbital follow camera.

Source: Unity Technologies (Starter Assets Third Person Character Controller).

Credit / License: Unity Companion License / Asset Store Terms. Free for commercial and non-commercial compiled builds; requires standard Unity engine inclusion in game credits.

Protagonist Model & Rigging:

What is Reusable: The Humanoid bone hierarchy, animation controller mapping, and custom "NaN" mesh.

Source: Adobe Mixamo (Erika_Archer base mesh & standard Humanoid locomotion clips).

Credit / License: Royalty-free under Adobe Mixamo Terms of Service for use in embedded video game builds (attribution to Adobe Mixamo in credits; raw .fbx files cannot be redistributed open-source).

Modular Room & Corridor Prefab Kit:

What is Reusable: Floor, wall, and doorway prefabs that can be re-skinned into decaying Backrooms/liminal spaces for Act 3 using shader/material swaps.

Source: Custom-modeled in Blender / Unity primitives.

Credit / License: Original creator IP 100% owned, no third-party attribution required.

"Stimuli" Trigger & Room Stabilization Scripts:

What is Reusable: The event-driven C# interaction architecture (StimulusItem.cs, RoomStabilizer.cs) can power every memory puzzle across all remaining chapters.

Source: Custom original C# codebase.

Credit / License: Original creator IP fully reusable across future episodes or spinoff projects.

Audio & Distortion Cues:

What is Reusable: Heartbeat pulses, hospital ambient drone, and the "Static" entity audio stems.

Source: Pixabay / FreeSound.org.

Credit / License: Pixabay Content License / Creative Commons (CC-BY 4.0 / CC0). Requires listing contributing sound artist handles in the game's final end-credits roll.

Question / Next Step:

Should I set up an automated Credits.txt file inside /Requirements/ now so that attribution for new sound stems and models is recorded the moment they are downloaded?

boardNotes

I will continue to work on the small project until I am ready to move up to the team project.

Whiteboard

Diagram: StimuLye vertical slice plan (solo, Unity 6.6, AAA framework)

The mission is a playable vertical slice: a dark opening, one looping ward corridor, and one Stimuli memory room. I'm working solo, so each box shows the "hat" I wear.

Stage 1 (Application, Technical hat): set up the project with Unity 6.6 + URP and the Third Person Starter Assets so I have working movement immediately. I also create the folders and Project_LOG.txt. Test: WASD, jump, and camera work.

Stage 2 (Action, 3D Artist hat): import the Mixamo NaN model, set it to Humanoid, unpack PlayerArmature, and swap the geometry. Test: it runs and jumps with no pink shaders.

Stage 3 (Action, Level Designer hat): build modular wall, floor, and door prefabs with colliders and dark fog. Test: no clipping or falling through floors.

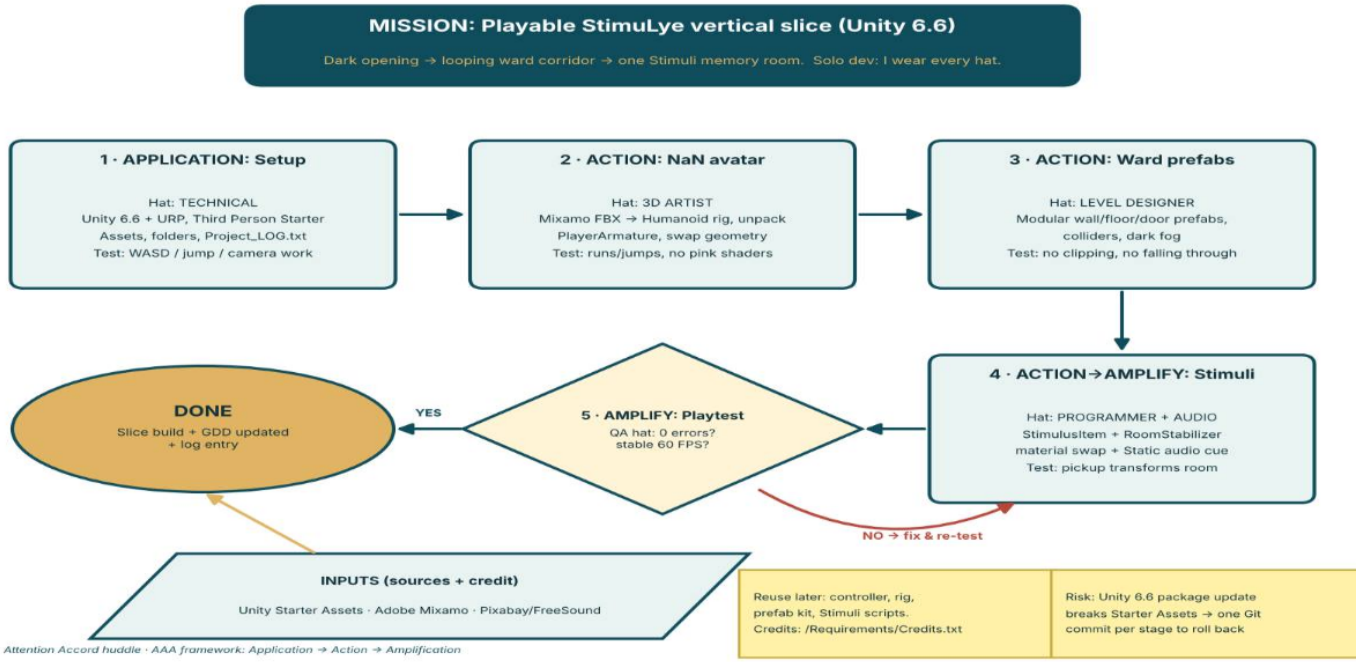
Stage 4 (Action to Amplification, Programmer + Audio hat): StimulusItem triggers RoomStabilizer, which swaps dark materials for hospital ones and plays the Static audio cue. Test: pickup visibly transforms the room.

Stage 5 (Amplification, QA hat): the decision diamond. If the playtest shows errors or low FPS, I loop back to stage 4 and fix. If it is clean (0 errors, stable 60 FPS), the slice is Done and I update the GDD and log.

Inputs are Unity Starter Assets, Adobe Mixamo, and Pixabay/FreeSound. Every credit goes in /Requirements/Credits.txt so these parts can be reused in later acts.

Risk: a Unity 6.6 package update could break Starter Assets, so I make one Git commit per stage to roll back.

CS110 / INDEPENDENCE ROAD



Whiteboard

Production Pipeline Explanation: StimuLye Vertical Slice (AAA Framework)

Our production pipeline applies Dr. Malik Stalbert's AAA Framework (Application Action Amplification) to transition our team smoothly from an initial template into a playable, narrative-driven prototype:

1. Stage 1: Application (Setup & Baseline)

Following the principle that "application gets them moving," we do not start by writing code from scratch. We establish our shallow directory structure (D:\PROJECTS\StimuLye) to bypass Windows path length limits and deploy Unity's verified Third Person Core template. This gives us immediate physics, camera orbit, and movement controls within 30 minutes.

2. Stages 2 & 3: Action (Mechanics, Avatars & Environments)

Once the baseline is alive, the team enters the "Action" phase, where concepts and custom assets are introduced:

- Character Integration: We import our NaN model, set its rig to Humanoid, safely unpack the player armature, and swap the geometry. The Humanoid avatar acts as the bridge so existing template animations retarget cleanly to our custom protagonist.
- Environment Construction: Our level designer uses modular prefabs (the "cookie-cutter" approach) to build the looping hospital ward and dark memory rooms without wasting memory.

3. Stage 4 & 5: Amplification (Core Loop, Polish & Verification)

In the "Amplification" phase, private technical competence expands into an integrated experience:

- Core Narrative Mechanic: We implement the "Stimuli" interaction script. Picking up the sensory object triggers a room material swap from dark void to stable hospital textures while playing the first "Static" audio cue.

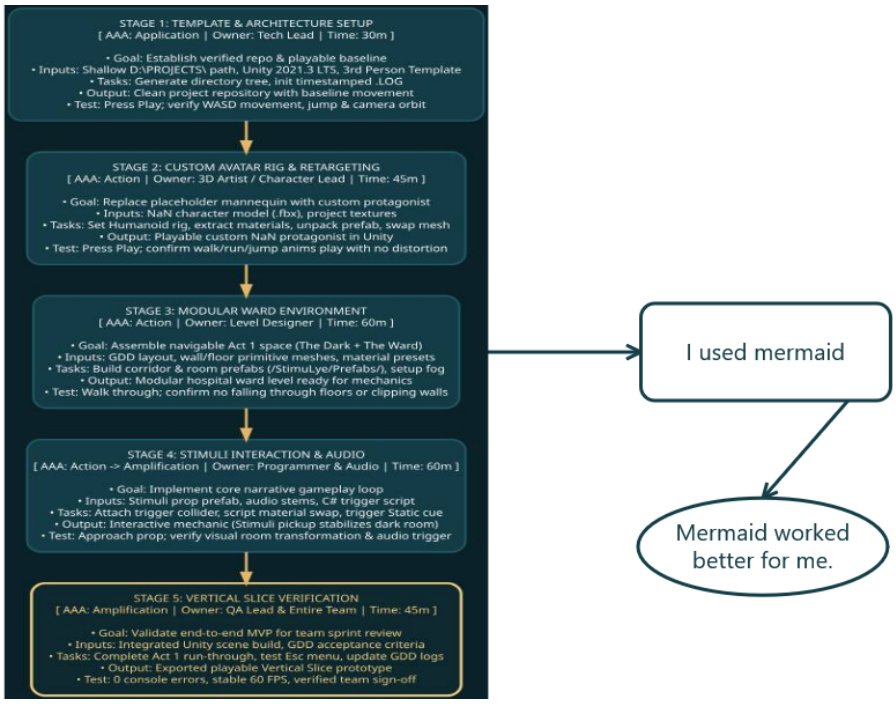
CS110 / INDEPENDENCE ROAD

- Verification & Team Review: The team conducts an end-to-end playtest, verifying that the opening dark sequence, corridor movement, and memory stabilization run smoothly with zero console errors.

Roles & Time Estimates:

- Block 1 (30m): Technical Lead (Template & Directories)
- Block 2 (45m): 3D Artist / Character Lead (Rigging & Retargeting)
- Block 3 (60m): Level Designer (Modular Ward Prefabs)
- Block 4 (60m): Gameplay Programmer & Audio Lead (Stimuli Triggers)
- Block 5 (45m): QA & Entire Team (Playtest & Milestone Sign-off)

Total Sprint Duration: 4 Hours (240 Minutes)



codeTables

0

id

061b1987-dcfd-45cf-80bc-0948d0a0ace7

title

Classes & components

columns

0

label

Class

type

text

1

CS110 / INDEPENDENCE ROAD

label

Responsibility

type

text

2

label

GameObject

type

text

rows

0

0

ThirdPersonController

1

Processes movement calculations, sprint, jump physics, gravity, and drives animator parameters.

2

PlayerArmature

1

0

StarterAssetsInputs

1

Reads raw input data (WASD, mouse delta, space, shift) from the Input System and stores state flags.

2

PlayerArmature

2

0

CharacterController

1

Unity engine physics component that manages capsule collision, ground detection, and slopes.

2

PlayerArmature

3

0

StimulusItem

1

Detects player proximity, triggers interaction events, and signals memory recovery when collected.

2

Stimulus_MemoryObject

4

CS110 / INDEPENDENCE ROAD

0

RoomStabilizer

1

Listens for Stimulus collection; swaps room shaders/materials from dark void to hospital textures.

2

Environment_Manager

5

0

StaticController

1

Monitors player distance to boundaries; increases audio distortion and visual grain as "Static" stalks.

2

Static_Antagonist

6

0

PauseMenuController

1

Listens for the Esc key; pauses time and displays the CRT power-down / heart monitor scene.

2

UI_Canvas_Manager

1

id

6a62977a-857f-4e05-b30d-ec5a07798942

title

Variables / properties

columns

0

label

Name : type

type

text

1

label

Value / purpose

type

text

2

label

Used by

CS110 / INDEPENDENCE ROAD

type

text

rows

0

0

MoveSpeed

1

Defines base walk velocity vs. sprint speed in the ward.

2

ThirdPersonController

1

0

_animIDSpeed

1

Cached hash IDs for fast Animator parameter updates.

2

ThirdPersonController

2

0

isStabilized

1

Prevents the room transformation from firing more than once.

2

RoomStabilizer

3

0

darkMaterial

1

References to the low-poly void shader vs. lit hospital texture.

2

RoomStabilizer

4

0

staticProximityThreshold

1

Distance from player before static distortion audio begins.

2

StaticController

5

CS110 / INDEPENDENCE ROAD

0

stimulusSound

1

Audio stems played on memory pickup and enemy approach.

2

AudioManager

2

id

595bbdd0-1eb9-4603-a8b8-c8d31f390b23

title

f(x) Functions

columns

0

label

Function

type

text

1

label

Inputs result

type

text

2

label

When called

type

text

rows

0

0

Awake()

1

Engine Initialization

2

Caches component references (GetComponent<Animator>(), CharacterController) before scripts run.

1

0

Start()

1

CS110 / INDEPENDENCE ROAD

First Frame

2

Sets the opening scene state: turns off ward lights, applies black void materials, locks cursor.

2

0

Update()

1

Every Frame

2

Polls player movement vectors, updates camera angles, and checks for the Escape key input.

3

0

OnTriggerEnter(Collider)

1

Physics Collision Event

2

Detects when NaN steps into the StimulusItem radius or enters a "Static" trigger volume.

4

0

OnFootstep(AnimationEvent)

1

Animation Event

2

Plays footstep sound cues synchronized with the walk/run animation clip frames.

3

id

b0f46e68-aa40-4beb-a479-052e7cd5204d

title

Packages / using

columns

0

label

Package / namespace

type

text

1

label

Why included

type

CS110 / INDEPENDENCE ROAD

text

2

label

Tested

type

checkbox

rows

0

0

using UnityEngine;

1

The fundamental foundation of all Unity code. Provides access to MonoBehaviour, GameObject, Transform, Vector3, Time, and core physics engine types.

2

true

1

0

using UnityEngine.InputSystem;

1

Replaces the legacy Input.GetKey system. Required in Unity 6 for cross-platform event-based input handling (keyboard, mouse look, and gamepads) with zero frame-rate polling lag.

2

true

2

0

using Cinemachine;

1

Controls the dynamic third-person follow camera rig. Automatically handles procedural damping, collision detection with ward walls, and smooth cursor orbital movement.

2

true

3

0

using UnityEngine.Events;

1

Allows decoupled communication via UnityEvent or Action. When NaN collects a Stimulus, the item simply raises an event; the RoomStabilizer and AudioSource respond without being hardcoded to each other.

2

true

4

CS110 / INDEPENDENCE ROAD

0

using UnityEngine.Rendering.Universal;

1

Gives C# runtime control over Unity 6's Universal Render Pipeline (URP) volume components (vignette, film grain, chromatic aberration) to create the visual "Static" distortion effect.

2

true

communicationTables

0

id

5a24f4fd-3441-4b83-8102-50851844f53a

title

Team check-ins

columns

0

label

Description / contact / owner

type

text

1

label

When Pacific

type

datetime-local

2

label

Agreed

type

checkbox

rows

0

0

Lexxie

1

2

true

Task list

0

CS110 / INDEPENDENCE ROAD

id

75169224-740e-41a8-8b8a-24a6137bee77

task

Documentation

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

1

id

de1342be-2e15-45b8-a532-e54167c65a25

task

Setup

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

2

id

ea92e30c-008a-4533-9ce6-e2786e733ed4

task

Player import

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

3

CS110 / INDEPENDENCE ROAD

id

3ba2fd21-ccbe-4d2d-8b62-3282e36cbb0b

task

Testing

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

4

id

66088a36-ec28-4a94-a6d7-d33cb121cf18

task

Handoff

Task owner reference

Lexxie

Deadline

2026-11-12T20:30

Completed

false

Execution / result

revision

1